Module 4 — DBMS, database design, transactions and performance

Learning outcomes

Explain how a relational database protects data, handles concurrency and executes queries; design a normalised schema; select indexes thoughtfully.

1. Normalisation and functional dependencies

A functional dependency A $\rightarrow$ B means A determines B. Normalisation reduces update, insertion and deletion anomalies.

• 1NF: attributes hold atomic values; no repeating groups.

• 2NF: 1NF and no non-key attribute depends on only part of a candidate key.

• 3NF: 2NF and no transitive non-key dependency on a key.

• BCNF: every determinant is a candidate key.

Example: `Enrollment(student_id, student_name, course_id, course_name, grade)`. Student name depends on student_id; course name depends on course_id; grade depends on the combination. Split into Students, Courses and Enrollments.

Denormalisation is a deliberate read-performance or analytical modelling choice, with consistency cost.

2. Transactions and ACID

A transaction groups operations into one logical unit.

• Atomicity: all or none.

• Consistency: constraints/invariants hold.

• Isolation: concurrent work obeys the configured isolation guarantee.

• Durability: committed work survives failures according to system guarantees.

```
BEGIN;
UPDATE wallets SET balance = balance - 250 WHERE wallet_id = 10;
UPDATE wallets SET balance = balance + 250 WHERE wallet_id = 20;
COMMIT;
```

If the second update fails, rollback the transaction. A production transfer should also validate sufficient balance and affected row counts.

3. Isolation anomalies

• Dirty read: read uncommitted data.

• Non-repeatable read: same row returns a different committed value on reread.

• Phantom: predicate query returns a changed set.

• Lost update: one concurrent write overwrites another.

Standard isolation labels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Exact guarantees differ by engine. PostgreSQL uses MVCC and does not implement READ UNCOMMITTED as a distinct weaker mode; do not assume identical behaviour across databases.

4. Locks and deadlocks

Shared/exclusive lock concepts protect reads/writes; actual lock modes differ by engine. A deadlock is a cycle of transactions waiting on resources held by each other. The database aborts a victim. Reduce risk with short transactions, consistent lock order, suitable indexes and safe retry.

5. Indexes

B-tree indexes commonly support equality and range lookups. Composite index order matters. Covering indexes may satisfy a query from index data alone. Indexes consume space and add write overhead. A low-selectivity index may not help much; use plans and workload evidence.

6. Query planning

An optimiser estimates cost and selects a plan. Understand sequential scan, index scan, nested-loop join, hash join, merge join, sort and aggregate. Compare estimated and actual rows. `EXPLAIN ANALYZE` executes the query; use caution on expensive or modifying statements.

7. Stored routines and database objects

• View: stored query definition.

• Materialized view: persisted query result requiring refresh/maintenance.

• Stored procedure: invoked server-side routine; may contain multiple statements and procedural logic.

• Function: returns scalar/table value and may be callable within expressions, subject to engine restrictions.

• Trigger: fires automatically on configured table events.

PostgreSQL procedure example:

```
CREATE OR REPLACE PROCEDURE mark_expired_subscriptions(p_cutoff date)
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE subscriptions
  SET status = 'EXPIRED'
  WHERE end_date < p_cutoff
    AND status = 'ACTIVE';
END;
$$;

CALL mark_expired_subscriptions(DATE '2026-01-01');
```

This syntax is PostgreSQL-specific. MySQL and SQL Server syntax and transaction restrictions differ.

8. DELETE, TRUNCATE and DROP

• DELETE removes selected rows; can use WHERE.

• TRUNCATE removes all rows efficiently in many engines; identity/transaction behaviour differs.

• DROP removes the object definition and data.

Know the target engine's transactional and permission semantics.

9. Partitioning, replication, sharding and backup

Partitioning divides a logical table into partitions; partition pruning can reduce scans. Replication keeps copies for availability/read scaling but may lag. Sharding distributes data across nodes and complicates cross-shard queries. Backup provides recoverability; replication alone is not a backup.

Guided lab

Create:

```
CREATE TABLE students (
  student_id BIGINT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT NOT NULL
);
CREATE TABLE courses (
  course_id BIGINT PRIMARY KEY,
  course_name TEXT NOT NULL
);
CREATE TABLE enrollments (
  student_id BIGINT REFERENCES students(student_id),
  course_id BIGINT REFERENCES courses(course_id),
  enrolled_on DATE NOT NULL,
  grade NUMERIC(5,2) CHECK (grade BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id)
);
```

Tasks:

1. Insert valid rows and demonstrate a foreign-key failure.

2. Try duplicate email and explain the constraint error.

3. Start a transaction, update a grade and rollback.

4. Add an index for course enrolment lookup.

5. Compare an EXPLAIN plan before and after, noting that a tiny table may still use a sequential scan.

6. Create a view showing course enrolment counts.

Lab answers / reasoning

• The composite primary key prevents a student enrolling in the same course twice.

• The foreign key prevents enrolment for a non-existent student/course.

• Rollback restores the previous committed state.

• An index is not guaranteed to be selected; table size and selectivity matter.

• A view stores the query definition, not necessarily the result.

Independent DBMS questions

1. Why is a unique constraint different from a primary key?

2. Why can a transaction be consistent but not isolated enough for a business rule?

3. What happens if an application retries a transfer after a timeout but the first commit actually succeeded?

4. Why can a trigger make debugging harder?

5. Why can a replica return stale data?

6. When is denormalisation justified?

Answer guide

1. A table may have multiple unique candidate keys but one primary key; NULL handling differs by engine.

2. Two individually valid operations can interleave and violate a cross-row invariant without suitable isolation/locking.

3. Use an idempotency key or unique transaction reference; do not blindly repeat a non-idempotent debit.

4. Side effects happen implicitly during DML and may be hidden from application flow.

5. Asynchronous replication lag.

6. Read-heavy analytical/reporting use cases where measured benefit outweighs update complexity.

External practice for this module

This module needs conceptual recall and hands-on database work; coding sites alone are not sufficient.

1. PostgreSQL documentation — constraints — read primary keys, unique, foreign keys and CHECK constraints; reproduce the lab in PostgreSQL.

2. PostgreSQL documentation — transactions and transaction isolation — compare isolation levels and reproduce a two-session concurrency example.

3. PostgreSQL documentation — indexes — create/drop B-tree indexes and compare plans.

4. Use The Index, Luke — complete the introductory chapters on indexes, composite indexes and execution plans.

5. GeeksforGeeks DBMS practice — use the MCQs for quick recall of keys, normalisation, transactions and concurrency; verify engine-specific claims against PostgreSQL documentation.

Hands-on assignment: create a small schema, demonstrate a constraint violation, commit and rollback, run two concurrent sessions, create an index and compare `EXPLAIN ANALYZE`. Keep screenshots or SQL scripts as evidence.

Exit requirement: answer 25 DBMS oral questions without notes and explain one anomaly, one deadlock scenario, one index trade-off and one idempotent transaction retry.